

南開大學
Nankai University

计算机学院

并行程序设计 **Lab4** 报告

MPI 多进程编程实验

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 6 月 5 日

项目仓库：https://github.com/Aurora1094/NKU-ParallelProgramming

摘　要

本实验选择近似最近邻搜索（Approximate Nearest Neighbor Search, ANN）作为 MPI 并行优化对象，围绕 DEEP100K 数据集上的高维向量查询任务，在前序 SIMD 与 OpenMP 优化基础上进一步实现并比较多种 MPI 多进程并行搜索方案。实验以 MPI-IVF-SIMD 为主线，扩展到 batch 通信优化、MPI + OpenMP + SIMD 混合并行、通信 API 与负载均衡优化、MPI-IVF-PQ 压缩索引、MPI shard-HNSW 图索引以及 IVF-HNSW 组合索引等多个方向，重点分析不同并行策略下的 Recall-Latency 权衡关系。

本文首先实现了 MPI-IVF-SIMD 基线框架：将 base 向量按照 MPI rank 划分，每个进程在本地分片上构建 IVF 索引；查询阶段由 root 进程广播 query batch，各 rank 独立执行本地 IVF-SIMD 搜索并返回 local top-k，最后由 root 完成 global top-k 合并。扩展性实验表明，MPI 数据划分能够显著降低本地搜索规模，使查询延迟从单进程配置的 1171.103 μs 降低到 $np = 16$ 时的 285.984 μs，获得 4.10× 端到端加速。但当进程数继续增大时，通信、同步和候选合并开销逐渐上升，限制了进一步扩展。

在通信优化方面，本文将逐 query 广播改为 batch query 广播。实验结果表明，batch size 从 1 增大到 64 后，MPI-IVF-SIMD 的平均查询延迟从 357.7 μs 降低到 101.3 μs，端到端加速比达到 3.53×，而 Recall@10 保持不变，说明 batch 通信能够有效减少 collective 调用次数和同步等待开销。进一步的混合并行实验显示，在固定总核数为 8 的条件下，纯 MPI 配置 $(8, 1)$ 的性能优于 $(4, 2)$、$(2, 4)$ 和 $(1, 8)$ 等 MPI + OpenMP 组合，说明在当前数据规模下，保持更多 MPI rank 以减小每个进程的本地分片，比增加 rank 内线程数更加有效。

本文还进一步探索了压缩索引和图索引的 MPI 并行化。MPI-IVF-PQ 通过 PQ 粗排和 SIMD rerank 降低候选扫描开销，在允许一定 Recall 损失的情况下能够获得更低延迟；例如高精度配置下 Recall@10 达到 0.93660，平均延迟为 133.399 μs。MPI shard-HNSW 则在高 Recall 区域表现突出，$np = 8, efSearch = 50$ 时 Recall@10 达到 0.99765，平均延迟为 205.815 μs，适合对精度要求更高的场景。相比之下，IVF-HNSW 的 Recall 主要受 $nprobe$ 限制，而 centroid-HNSW + IVF-HNSW 在当前 $nlist = 256$ 的规模下由于额外图搜索开销较大，未带来明显收益。

综合实验结果表明，ANN 查询的 MPI 并行性能由索引结构、数据划分、通信粒度、负载均衡和参数选择共同决定。MPI 数据分片是降低查询延迟的主要来源，batch 通信是本实验中收益最明显的通信优化；而非阻塞通信和简单负载均衡策略收益相对有限。对于低延迟场景，可以优先考虑 batch 优化后的 MPI-IVF-SIMD 或 MPI-IVF-PQ；对于高 Recall 场景，MPI shard-HNSW 更具优势。dense 参数网格实验进一步展示了不同方法的 Pareto frontier，为选择合适的 Recall-Latency 折中点提供了依据。

关键词： MPI；IVF；IVF-PQ；HNSW；Recall-Latency

目录

1 前言 **1**
1.1 实验环境 . . . 1
1.2 实验内容概述 . . . 1
1.3 实验数据与评价指标 . . . 2

2 算法原理与实现 **3**
2.1 MPI 数据划分与并行搜索框架 . . . 3
2.2 IVF-SIMD 索引构建与查询 . . . 4
2.3 MPI 通信、global top-k 合并与负载均衡 . . . 5
2.4 MPI + OpenMP + SIMD 混合并行 . . . 6
2.5 IVF-PQ 压缩编码与 rerank . . . 7
2.6 HNSW 图索引与 shard-HNSW 并行 . . . 8
2.7 IVF-HNSW 与 centroid-HNSW 组合索引 . . . 9

3 实验设计 **10**
3.1 MPI-IVF-SIMD 基线实验 . . . 11
3.2 batch 通信优化实验 . . . 11
3.3 MPI + OpenMP + SIMD 混合并行实验 . . . 12
3.4 通信 API 与负载均衡优化实验 . . . 12
3.5 MPI-IVF-PQ 压缩索引实验 . . . 12
3.6 MPI 图索引并行实验 . . . 12
3.7 HNSW 分片策略与节点布局实验 . . . 12
3.8 dense 参数网格实验 . . . 13

4 实验结果与分析 **13**
4.1 MPI-IVF 扩展性分析 . . . 13
4.2 MPI-IVF 优化分析 . . . 14
4.3 MPI-IVF-PQ 结果分析 . . . 15
4.4 MPI 图索引结果分析 . . . 16
4.5 参数网格与综合性能对比 . . . 17

5 总结 **19**
5.1 实验结论 . . . 19
5.2 不足与改进方向 . . . 20
5.3 个人心得体会 . . . 20

A 各阶段实验参数 **21**

1 前言

1.1 实验环境

本实验主要在 ARM/openEuler 服务器平台完成，并在 x86/WSL2 本地平台上进行补充对比测试。[1]采用 MPI 多进程编程模型对近似最近邻搜索（Approximate Nearest Neighbor Search, ANN）算法进行分布式并行优化。实验代码使用 C++ 编写，并结合 MPI、OpenMP 与 SIMD 实现多层次并行加速。部分单机基线和补充实验在 x86/WSL2 本地平台上完成，用于对比不同平台和实现方式下的性能差异。

实验平台的基本环境如表 1 所示。

表 1: 实验软硬件环境

项目	ARM/openEuler 服务器	x86/WSL2 本地平台
CPU 架构	aarch64	x86_64
CPU 型号	Kunpeng-920	Intel Core i9-14900HX
核心/线程	8 cores	32 logical CPUs
操作系统	openEuler 22.03 LTS-SP4	Ubuntu 22.04 WSL2
编译器	g++ 10.3.1 / mpic++	g++ 11.4.0
SIMD 指令集	NEON / ASIMD	AVX2 / FMA
内存	30 GiB	15 GiB
并行框架	MPI / OpenMP / SIMD	OpenMP / SIMD

MPI 实验通过 `mpic++` 编译，并使用 `qsub` 或 `mpiexec` 在服务器上运行。实验中根据节点资源设置 `nodes`、`ppn` 和 `np`，并保证

$$\texttt{np} \le \texttt{nodes} \times \texttt{ppn}.$$

与前序实验相比，本实验的重点从单机 SIMD/OpenMP 优化扩展到 MPI 分布式搜索，因此需要重点关注数据划分、query 广播、局部搜索、候选收集、全局 top-k 合并以及通信开销。

1.2 实验内容概述

本实验围绕 DEEP100K 数据集上的 ANN 查询任务展开，目标是在保持较高召回率的前提下降低查询延迟，并分析不同 MPI 并行策略对性能的影响。实验在已有 SIMD、OpenMP、IVF、IVF-PQ 和 HNSW 实现基础上，进一步完成 MPI 多进程迁移、混合并行优化、通信优化和图索引并行实验。

本文主要完成以下几类实验：

1. **MPI-IVF-SIMD 基线实验**。将 IVF-SIMD 迁移到 MPI 多进程环境中，按照 rank 划分 base 向量，各进程构建本地 IVF 索引并执行局部搜索，最后由 root 进程完成候选收集与 global top-k merge。

2. **MPI 扩展性与通信优化实验**。测试不同进程数下的 wall latency、speedup、parallel efficiency 和 Recall，并进一步比较不同 batch size、blocking/nonblocking 通信方式以及分阶段通信开销。

3. **MPI + OpenMP + SIMD 混合并行实验**。在 MPI 进程间划分数据的基础上，在 rank 内使用 OpenMP 处理 query batch，并继续使用 SIMD 加速距离计算。实验比较 $(8,1)$、$(4,2)$、$(2,4)$ 和 $(1,8)$ 等不同进程/线程组合。

[1]ARM/openEuler 平台为本实验主要 MPI 测试平台，x86/WSL2 平台主要用于补充验证和跨平台性能对比。

4. **IVF 负载均衡与节点布局实验**。对 contiguous、round-robin、greedy 等负载均衡策略以及不同节点布局进行测试，分析本地搜索时间、通信时间和跨节点运行方式对性能的影响。
5. **MPI-IVF-PQ 与图索引并行实验**。在 MPI-IVF 基础上引入 PQ 编码与 rerank，分析压缩索引的 recall-latency trade-off；同时实现 MPI shard-HNSW、IVF-HNSW 和 centroid-HNSW + IVF-HNSW 等图索引并行方案。
6. **dense 参数网格实验**。对 MPI-IVF-SIMD 和 MPI shard-HNSW 补充密集参数实验，系统考察 np、$nlist$、$nprobe$、efSearch 和分片策略等参数对 Recall 与 latency 的影响。

整体而言，本实验的主线是：先构建 MPI-IVF-SIMD 基线，再从通信、混合并行、负载均衡和节点布局等方面进行优化，随后扩展到 IVF-PQ 压缩索引和 HNSW 图索引，最后通过 dense 参数网格实验分析不同方法的 recall-latency trade-off。

1.3 实验数据与评价指标

本文统一使用 DEEP100K 数据集。该数据集包含 base 向量、query 向量和 ground truth 文件。其中 base 集合包含 100000 个 96 维 float32 向量，query 集合包含 10000 个 96 维 float32 查询向量。实验中通常选取前 2000 条 query 作为正式测试集，并使用 ground truth 计算近似搜索结果的召回率。

给定 base 向量集合

$$X = \{x_1, x_2, \ldots, x_N\}, \quad x_i \in \mathbb{R}^d,$$

以及查询向量

$$q \in \mathbb{R}^d,$$

ANN 的目标是在 X 中返回与 q 最相近的 k 个向量。由于近似搜索不一定返回精确 KNN 结果，因此需要同时评价搜索精度和查询效率。

本文主要采用以下指标：

1. **Recall@10**：衡量 ANN 返回结果与 ground truth 中真实前 10 近邻的重合程度，定义为

$$Recall@10 = \frac{|KNN_{10}(q) \cap ANN_{10}(q)|}{10}.$$

2. **Wall Latency**：查询阶段端到端墙钟时间除以 query 数量，单位为 $\mu s/query$，定义为

$$Wall\ Latency = \frac{T_{wall}}{Q}.$$

该指标包含 MPI 通信、本地搜索、候选收集和全局合并等实际开销，是本文比较不同 MPI 并行策略的核心指标。[2]

3. **Speedup 与 Parallel Efficiency**：用于评价 MPI 并行扩展性，定义为

$$Speedup(p) = \frac{T_1}{T_p}, \qquad Efficiency(p) = \frac{Speedup(p)}{p}.$$

4. **阶段耗时**：包括 broadcast time、local search time、gather time、merge time、build time 等，用于分析通信开销、负载均衡和索引构建代价。

[2]该指标不包含数据加载和索引构建时间；索引构建开销以 build time 单独列出。

2 算法原理与实现

相比 OpenMP 的共享内存模型，MPI 属于分布式内存编程，使用显式消息传递进行进程间通信，因此在编写 MPI 程序时需特别注意数据的划分与分布、通信顺序和死锁风险、进程同步以及通信开销优化（如合并小消息、使用非阻塞通信、合理利用集体操作）。我们在实现中需要格外注意这一点。

2.1 MPI 数据划分与并行搜索框架

算法简介：在单机 ANN 搜索中，所有 base 向量通常存储于同一进程中，查询时由一个进程完成完整搜索。MPI 并行化的核心思想是将 base 集合划分到多个 MPI rank 中，每个 rank 只维护自己的本地分片，并在查询时并行搜索本地索引。设 base 向量集合为

$$X = \{x_1, x_2, \ldots, x_N\}, \quad x_i \in \mathbb{R}^d,$$

MPI 进程数为 P，则可将 X 划分为

$$X = X_0 \cup X_1 \cup \cdots \cup X_{P-1}, \quad X_r \cap X_s = \varnothing \ (r \neq s),$$

其中 X_r 表示 rank r 负责的本地 base 分片。

查询阶段，root rank 读取 query 或 query batch，并通过 MPI 广播给所有 rank。每个 rank 在本地索引上搜索得到 local top-k 候选集合

$$C_r(q) = \{(id, dist)\}_r^k,$$

然后通过 `MPI_Gather` 或类似通信方式将各 rank 的候选集合收集到 root rank。最终 root rank 对

$$C(q) = \bigcup_{r=0}^{P-1} C_r(q)$$

执行 global top-k 合并，得到最终 ANN 结果。

Algorithm 1 MPI 并行 ANN 搜索框架

Require: base 向量集合 X，查询集合 Q，MPI 进程数 P，返回规模 k
Ensure: 每个 query 的全局 top-k 近邻结果

```
MPI 初始化，获得当前进程编号 rank 与总进程数 P
按 rank 将 base 向量划分为本地分片 X_rank
每个 rank 在 X_rank 上构建本地索引 I_rank
for 每个 query batch B do
    root rank 读取 query batch B
    使用 MPI_Bcast 将 B 广播到所有 rank
    for batch 中每个查询向量 q do
        每个 rank 在本地索引 I_rank 上搜索，得到 local top-k 候选 C_rank(q)
    end for
    使用 MPI_Gather 收集所有 rank 的候选结果
    root rank 对所有候选执行 global top-k merge
    root rank 计算 Recall 与查询耗时
end for
MPI 结束 =0
```

设计动机： MPI 数据划分可以使每个进程只处理一部分 base 向量，从而降低单个 rank 的本地搜索规模。理论上，当数据划分较均衡且通信开销较小时，随着进程数增加，查询延迟应明显下降。

性能优化： MPI 并行搜索的性能由三部分共同决定：本地搜索时间、通信时间和 root 合并时间。其中本地搜索时间通常占主要部分，通信时间主要来自 query broadcast 与候选 gather，root 合并时间则随 $P \times k$ 的候选数量增加而增加。

并行化点： 本文的 MPI 并行主要发生在 base 数据划分和本地搜索阶段。每个 rank 独立执行本地 ANN 搜索，只有 query 广播、候选收集和 global top-k merge 阶段需要进程间通信。因此，该框架具有较好的可扩展性，但也需要重点处理通信开销和负载均衡问题。

2.2 IVF-SIMD 索引构建与查询

算法简介： IVF（Inverted File）通过聚类将向量空间划分为多个区域。首先对 base 集合执行 k-means 聚类，得到 $nlist$ 个聚类中心

$$\{c_1, c_2, \ldots, c_{nlist}\}.$$

随后将每个 base 向量分配到距离最近的聚类中心所对应的倒排表中。设第 j 个倒排表为 L_j，则有

$$x_i \in L_j, \quad j = \arg\min_t \|x_i - c_t\|_2.$$

查询时，给定 query 向量 q，先计算其与所有聚类中心的相似度或距离：

$$d_j = \|q - c_j\|_2, \quad j = 1, \ldots, nlist.$$

随后选择距离最近的 $nprobe$ 个聚类中心，其对应倒排表集合记为

$$S = \mathrm{Top}_{nprobe}\{-d_j\}.$$

最后只扫描这些倒排表中的候选向量，并维护 top-k 结果。

在本文实现中，IVF 的候选向量距离计算使用 SIMD 加速。对于 96 维 float32 向量，SIMD 可以一次处理多个浮点元素，从而降低内积或 L2 距离计算的循环开销。

Algorithm 2 IVF-SIMD 查询过程

Require: 查询向量 q，聚类中心 $\{c_j\}_{j=1}^{nlist}$，倒排表 $\{L_j\}_{j=1}^{nlist}$，参数 $nprobe$，返回规模 k
Ensure: 查询向量 q 的 top-k 近邻结果
初始化结果堆 H
for $j = 1$ to $nlist$ **do**
  使用 SIMD 计算 $d_j = \|q - c_j\|_2$
end for
选择距离最近的 $nprobe$ 个聚类中心，记其倒排表集合为 S
for 每个倒排表 $L_j \in S$ **do**
  for 每个候选向量 $x_i \in L_j$ **do**
    使用 SIMD 计算 $dist = \|q - x_i\|_2$
    用 $(id_i, dist)$ 更新 top-k 结果堆 H
  end for
end for
return H =0

设计动机： Flat scan 需要对每个 query 遍历所有 base 向量，计算量为 $O(Nd)$。IVF 通过只搜索与 query 最相关的若干倒排表，将搜索范围从全体 base 向量缩小到候选子集，从而显著降低查询延迟。

性能优化： IVF 的性能主要受 *nlist* 和 *nprobe* 影响。*nlist* 越大，每个倒排表平均长度越短，但聚类中心数量增加会带来更多中心距离计算；*nprobe* 越大，搜索的倒排表越多，Recall 通常越高，但 latency 也会增加。因此 IVF 的核心是平衡 Recall 与 latency。

并行化点： 在 MPI-IVF 中，每个 rank 只对本地 base 分片构建 IVF 索引，并在查询时只扫描本地倒排表。由于不同 rank 的搜索过程互相独立，因此可以天然并行。SIMD 进一步加速每个 rank 内部的距离计算。

2.3 MPI 通信、global top-k 合并与负载均衡

通信流程： MPI-IVF 查询过程中主要包含三类通信：

1. **Query broadcast：** root rank 将 query 或 query batch 广播到所有 rank；
2. **Candidate gather：** 每个 rank 将 local top-k 候选发送回 root；
3. **Global merge：** root 对所有候选结果进行全局 top-k 合并。

对于单个 query，若每个 rank 返回 k 个候选，则 root 需要合并的候选数量为

$$P \times k.$$

因此 global merge 的开销与 MPI 进程数 P 和返回规模 k 有关。

Algorithm 3 Global top-k 合并过程

Require: 来自 P 个 rank 的候选集合 $\{C_0, C_1, \ldots, C_{P-1}\}$，返回规模 k
Ensure: 全局 top-k 结果 G

```
初始化最大堆 G
for r = 0 to P − 1 do
    for 每个候选 (id, dist) ∈ C_r do
        if |G| < k then
            将 (id, dist) 插入 G
        else if dist 小于 G 中最大距离 then
            弹出当前最大距离元素
            插入 (id, dist)
        end if
    end for
end for
return G =0
```

batch 通信优化： 若每次只广播一个 query，则 MPI 调用次数较多，通信同步开销明显。本文将多个 query 组成 batch 后统一广播，使通信次数由

$$Q$$

降低为

$$\left\lceil \frac{Q}{B} \right\rceil,$$

其中 Q 为 query 总数，B 为 batch size。batch 广播可以有效降低频繁同步带来的开销。

负载均衡问题：在 MPI-IVF 中，即使每个 rank 分到的 base 数量相近，不同倒排表长度也可能存在差异。当某些 rank 被分配到更长的倒排表或更密集的候选区域时，其 local search time 会明显高于其他 rank。由于 global merge 必须等待所有 rank 完成本地搜索，因此总延迟近似受最慢 rank 控制：

$$T_{query} \approx \max_r T_{search}^{(r)} + T_{comm} + T_{merge}.$$

本文记录 max-rank search time 与 avg-rank search time，并测试 contiguous、round-robin、greedy 等划分方式，用于分析不同分片策略对负载均衡的影响。

设计动机： MPI 并行搜索不仅要减少每个 rank 的计算量，还要控制通信开销和最慢 rank 的拖尾影响。若数据划分不均衡，即使平均搜索量下降，整体 wall latency 仍可能被少数慢进程限制。

性能优化：本文从三个角度优化 MPI 查询框架：一是使用 batch broadcast 降低通信调用次数；二是比较 blocking 与 nonblocking 通信方式；三是通过不同分片策略改善 rank 间负载均衡。

2.4 MPI + OpenMP + SIMD 混合并行

算法简介： MPI 并行主要解决进程间数据划分和分布式搜索问题，而单个 rank 内部仍可能存在多个 CPU 核心可用。为进一步利用节点内并行资源，本文实现 MPI + OpenMP + SIMD 混合并行结构。

在该结构中，MPI 负责进程间 base 分片和候选结果通信；OpenMP 负责 rank 内多个 query 的并行处理；SIMD 负责单个 query 内部距离计算加速。整体并行层次为

$$\text{MPI ranks} \quad \rightarrow \quad \text{OpenMP threads} \quad \rightarrow \quad \text{SIMD lanes.}$$

为保证 MPI 线程安全，程序使用

```
MPI_Init_thread(..., MPI_THREAD_FUNNELED, ...)
```

初始化 MPI 环境，并规定只有主线程执行 MPI 通信操作。OpenMP 线程只负责本地 batch 中 query 的搜索。

Algorithm 4 MPI + OpenMP + SIMD 混合并行查询

Require: query batch B，本地索引 I_{rank}，返回规模 k
Ensure: 每个 query 的 local top-k 候选
1: 主线程调用 `MPI_Bcast` 接收 query batch B
2: 初始化本地候选缓冲区
3: 使用 OpenMP 并行遍历 batch 中的 query
4: **for** 每个 query $q \in B$ in parallel **do**
5: 当前线程在本地索引 I_{rank} 上执行 IVF 查询
6: 使用 SIMD 计算 query 与候选向量的距离
7: 为该 query 维护独立的 local top-k 堆
8: **end for**
9: 主线程调用 `MPI_Gather` 发送候选结果 =0

设计动机：当 MPI rank 数减少时，每个 rank 负责的 base 分片会变大，本地搜索开销随之增加。通过 OpenMP 在 rank 内并行处理 query batch，可以在一定程度上弥补 rank 数减少带来的本地计算压力。

性能优化： OpenMP 采用 query-level 并行，不同线程处理 batch 中的不同 query。由于每个 query 的搜索过程独立，并各自维护 top-k 结果，因此几乎不需要线程间同步，Recall 也基本一致。

并行化点： 混合并行由三层组成：MPI 在进程间划分 base 分片，OpenMP 在 rank 内并行处理 query batch，SIMD 加速单次距离计算。该结构能够同时利用进程级、线程级和指令级并行，但性能仍受分片规模、线程调度和内存带宽影响。

2.5 IVF-PQ 压缩编码与 rerank

算法简介： IVF-PQ 在 IVF 的基础上引入 Product Quantization，用压缩编码近似表示 base 向量，从而降低存储和距离计算开销。设原始向量维度为 d，PQ 将向量划分为 M 个子空间：

$$x = [x^{(1)}, x^{(2)}, \ldots, x^{(M)}].$$

每个子空间分别训练一个码本，码本大小为 K_s。对于第 m 个子空间，码本为

$$\mathcal{C}^{(m)} = \{c_1^{(m)}, c_2^{(m)}, \ldots, c_{K_s}^{(m)}\}.$$

向量 x 被编码为

$$code(x) = (a_1, a_2, \ldots, a_M),$$

其中 a_m 表示 $x^{(m)}$ 在第 m 个子空间中最近的码字编号。

查询时，先为 query 构建 lookup table：

$$LUT_m[t] = \|q^{(m)} - c_t^{(m)}\|_2,$$

则 query 与压缩向量 x 的近似距离为

$$\hat{d}(q, x) = \sum_{m=1}^{M} LUT_m[a_m].$$

IVF-PQ 先使用近似距离获得 top-p，再对这些候选使用 float 向量进行 SIMD rerank，得到最终 top-k。

Algorithm 5 MPI-IVF-PQ-SIMD 查询过程

Require: 查询向量 q，本地 IVF-PQ 索引，参数 $nprobe$，rerank 候选数 p，返回规模 k
Ensure: local top-k 候选

```
选择距离 q 最近的 nprobe 个 IVF 倒排表
根据 query q 构建 PQ lookup table
初始化 PQ 粗排候选堆 H_p
for 被选中倒排表中的每个 PQ code do
    使用 lookup table 计算近似距离 d̂(q, x)
    用近似距离更新 top-p 候选堆 H_p
end for
初始化精排堆 H_k
for 每个候选 x ∈ H_p do
    读取原始 float 向量
    使用 SIMD 计算真实距离 d(q, x)
    用真实距离更新 top-k 堆 H_k
end for
return H_k =0
```

设计动机： IVF-PQ 的目标是在降低访存与距离计算代价的同时尽量保持较高 Recall。PQ 粗排可以快速过滤大量候选，而 rerank 可以修正 PQ 近似距离带来的精度损失。

性能优化： *nprobe* 和 rerank_p 是 IVF-PQ 的两个关键参数。*nprobe* 越大，搜索范围越广，Recall 越高但 latency 增加；rerank_p 越大，参与精排的候选越多，Recall 提升但 SIMD float 距离计算开销增加。

并行化点： 在 MPI-IVF-PQ 中，每个 rank 仅对本地 base 分片进行 PQ 编码并构建本地 IVF-PQ 索引。查询阶段各 rank 独立执行 PQ 粗排和 SIMD rerank，最后 root rank 合并所有 local top-k 候选。与 MPI-IVF 相比，IVF-PQ 在本地搜索阶段降低了候选扫描成本，但增加了编码和 rerank 参数选择问题。

2.6 HNSW 图索引与 shard-HNSW 并行

算法简介： HNSW（Hierarchical Navigable Small World）是一种基于多层小世界图的 ANN 索引。其核心思想是在高层图中进行长距离快速跳转，在底层图中进行精细搜索。每个节点代表一个 base 向量，边连接近邻节点。查询时从入口点出发，在高层贪心下降，并在底层使用候选队列进行扩展搜索。

HNSW 的主要参数包括：

- M：每个节点最大连接边数；
- $efConstruction$：构建图时的候选搜索宽度；
- $efSearch$：查询时的候选搜索宽度。

$efSearch$ 越大，搜索范围越广，Recall 通常越高，但 latency 也随之增加。

Algorithm 6 HNSW 查询过程

Require: 查询向量 q，HNSW 图 G，入口点 e，参数 $efSearch$，返回规模 k
Ensure: top-k 近邻结果
从最高层入口点 e 开始
for 从最高层到第 1 层 **do**
  在当前层执行贪心搜索，更新距离 q 最近的入口点
end for
在第 0 层初始化候选队列与结果集合
while 候选队列非空且未达到停止条件 **do**
  取出当前最近候选节点 v
  for v 的每个邻居 u **do**
    计算 $dist(q, u)$
    若 u 更优，则加入候选队列与结果集合
  end for
  保留不超过 $efSearch$ 个候选
end while
从结果集合中返回距离最近的 top-k =0

MPI shard-HNSW： 本文首先实现 shard-HNSW 并行方案。该方案将 base 向量划分到不同 MPI rank，每个 rank 在本地分片上独立构建 HNSW 图。查询时 root rank 广播 query，各 rank 在本地图索引上搜索 local top-k，最后 root rank 收集候选并完成 global top-k merge。

Algorithm 7 MPI shard-HNSW 查询过程

Require: 查询向量 q，本地 HNSW 图 G_{rank}，参数 $efSearch$，返回规模 k
Ensure: 全局 top-k 近邻结果
1: root rank 广播查询向量 q
2: 每个 rank 在本地 HNSW 图 G_{rank} 上执行 HNSW 搜索
3: 每个 rank 得到 local top-k 候选
4: 使用 `MPI_Gather` 将候选发送到 root rank
5: root rank 执行 global top-k merge
6: **return** 全局 top-k 结果 =0

设计动机： HNSW 在单机上具有较高 Recall 和较低查询延迟，但完整 HNSW 图在多进程环境中不易直接跨 rank 维护。shard-HNSW 将图索引局部化，每个 rank 独立建图，避免复杂的跨进程图边访问，是实现 MPI 图索引并行的直接方案。

性能优化： shard-HNSW 的查询延迟受两个因素影响：一方面，MPI 分片使每个 rank 的图规模变小，本地搜索更快；另一方面，分片会使每个本地图只覆盖全局数据的一部分，可能影响候选质量，需要依赖 root merge 恢复全局 top-k。

并行化点： 每个 rank 的 HNSW 搜索完全独立，因此 shard-HNSW 的计算部分并行度较高。主要通信开销来自 query broadcast 和候选 gather。相比 MPI-IVF，HNSW 的本地搜索具有图遍历特征，其负载受图结构和 efSearch 影响。

2.7 IVF-HNSW 与 centroid-HNSW 组合索引

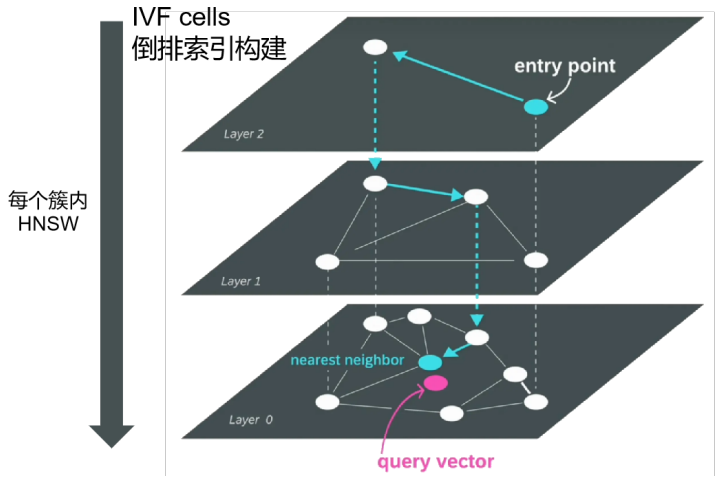


图 1: HNSW 多层图索引搜索过程示意图

算法简介： 为了结合 IVF 的粗粒度候选过滤能力和 HNSW 的高效图搜索能力，本文进一步实现 IVF-HNSW 组合索引。该方法首先使用 IVF 将 base 向量划分到不同 cluster 中，然后在每个 cluster 内建立一个局部 HNSW 图。查询时先选择 *nprobe* 个相关 cluster，再在对应 cluster 的 HNSW 图中搜索。

其查询过程可以表示为：

$$q \to \mathrm{Top}_{nprobe}(\{c_j\}) \to \{G_j\}_{j \in S} \to \textit{top-}k.$$

Algorithm 8 IVF-HNSW 查询过程

Require: 查询向量 q，聚类中心 $\{c_j\}$，cluster 内 HNSW 图 $\{G_j\}$，参数 $nprobe, efSearch, k$
Ensure: top-k 查询结果
1: 计算 q 与所有 IVF 聚类中心的距离
2: 选择最近的 $nprobe$ 个 cluster，记集合为 S
3: 初始化结果堆 H
4: **for** 每个 cluster $j \in S$ **do**
5: 在 cluster 内 HNSW 图 G_j 上执行 HNSW 搜索
6: 将局部候选结果插入结果堆 H
7: **end for**
8: 返回 H 中距离最近的 top-k =0

centroid-HNSW：在 IVF-HNSW 基础上，本文进一步尝试在 IVF 聚类中心上建立一层 centroid-HNSW，用 HNSW 图搜索代替暴力扫描所有 centroid。此时 query 不再直接遍历全部 $nlist$ 个中心，而是先在 centroid-HNSW 上搜索得到候选 cluster，再进入对应 cluster 内部的 HNSW 图进行搜索。

$$q \rightarrow G_{centroid} \rightarrow \{G_j\}_{j \in S} \rightarrow top\text{-}k.$$

Algorithm 9 centroid-HNSW + IVF-HNSW 查询过程

Require: 查询向量 q，centroid-HNSW 图 G_c，cluster 内 HNSW 图 $\{G_j\}$，参数 $nprobe, efSearch, k$
Ensure: top-k 查询结果
1: 在 centroid-HNSW 图 G_c 上搜索，得到候选 cluster 集合 S
2: 初始化结果堆 H
3: **for** 每个 cluster $j \in S$ **do**
4: 在 cluster 内 HNSW 图 G_j 中搜索候选向量
5: 将候选结果加入 H
6: **end for**
7: 返回全局 top-k 结果 =0

设计动机： IVF-HNSW 试图利用 IVF 减少需要访问的候选区域，再利用 HNSW 在每个区域内快速搜索。centroid-HNSW 则进一步希望降低选择 cluster 时扫描所有中心的开销。

性能优化： IVF-HNSW 的关键参数是 $nprobe$ 和 $efSearch$。$nprobe$ 控制进入多少个 cluster，$efSearch$ 控制每个 HNSW 图内搜索深度。二者增大通常会提高 Recall，但也会增加 latency。centroid-HNSW 是否有效取决于 centroid 数量和搜索开销：当 $nlist$ 不大时，直接扫描 centroid 的代价可能较低，此时额外引入 centroid-HNSW 反而可能带来负优化。

并行化点：在 MPI 环境中，IVF-HNSW 和 centroid-HNSW + IVF-HNSW 均可以沿用 shard 思路。每个 rank 维护本地 cluster 和本地图索引，查询广播后各 rank 独立搜索本地候选，再由 root 合并 global top-k。该方案保留了 MPI 分布式搜索框架，同时引入更复杂的层级索引结构，用于探索图索引与倒排索引结合后的性能表现。

3 实验设计

本章从实验问题出发设计测试方案整体实验流程如图 2 所示：首先基于 DEEP100K 数据集、ARM/openEuler 平台和 MPI + OpenMP + SIMD 环境构建实验输入；随后依次完成 MPI-IVF 基线、混合并行、IVF 通信与负载优化、MPI-IVF-PQ、MPI 图索引并行以及 dense 参数网格实验；最后统一从 Recall、Latency、Speedup 和 Parallel Efficiency 等指标对不同并行策略进行评价。

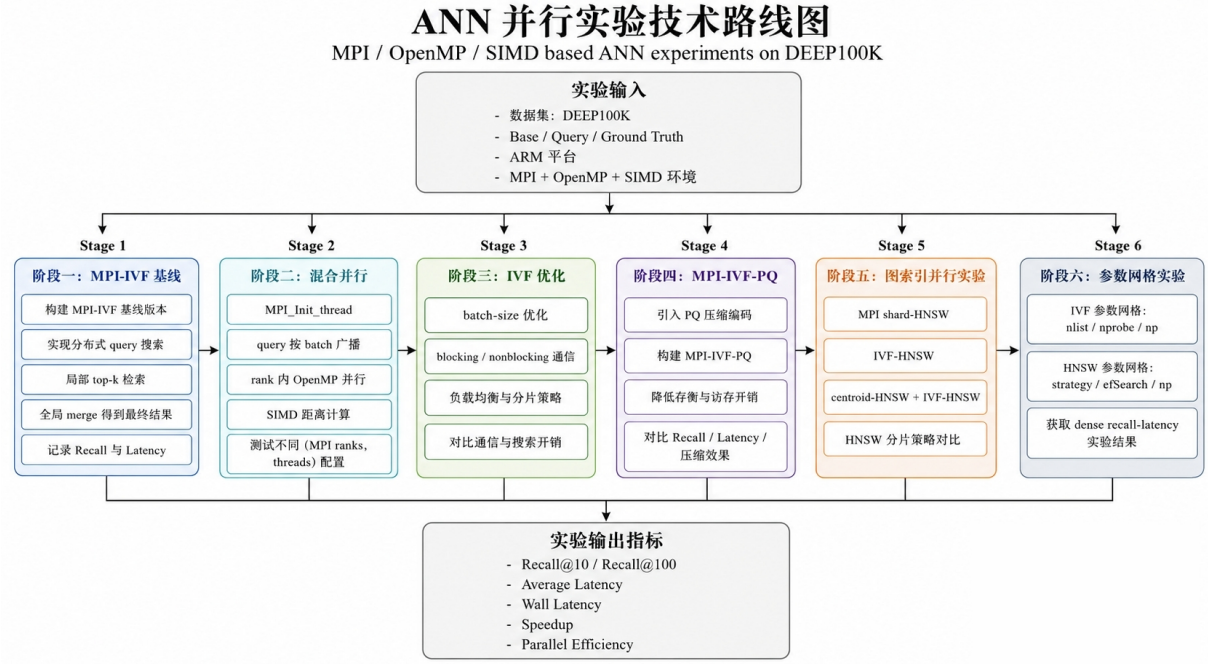


图 2: ANN 并行实验技术路线图

为了保证实验结果具有可比性，本文主要使用 DEEP100K 数据集，选取前 2000 条 query 进行测试，并对关键实验重复运行 3 次取平均值。实验过程中记录 Recall@10、wall latency、total wall time、build time、broadcast time、local search time、gather time 和 merge time 等指标，用于分析不同 MPI 并行策略的精度、效率和通信开销。

3.1 MPI-IVF-SIMD 基线实验

MPI-IVF-SIMD 基线实验用于验证 IVF-SIMD 在 MPI 多进程环境下的正确性和基本扩展性。实验中将 base 向量按照 MPI rank 划分，每个进程在本地分片上构建 IVF 索引；查询阶段由 root 进程广播 query batch，各 rank 独立执行本地 IVF-SIMD 搜索并返回 local top-k，最后由 root 完成 global top-k 合并。

该实验主要改变 MPI 进程数 np，测试 $np = 1, 2, 4, 8, 16$ 等配置下的 Recall、wall latency、speedup 和 parallel efficiency。通过该实验可以观察增加 MPI 进程后，本地搜索规模是否下降，以及通信和合并开销是否会限制进一步加速。

3.2 batch 通信优化实验

batch 通信优化实验用于分析 query 广播粒度对 MPI-IVF-SIMD 性能的影响。若每次只广播一个 query，MPI 通信调用次数较多，容易产生额外同步开销；因此本文将多个 query 组成 batch 后统一广播。

实验固定主要 IVF 参数和 MPI 进程数，比较不同 batch size 下的 broadcast time、gather time 和 wall latency。重点观察 batch size 从较小值增大到 64 时，通信开销是否明显下降，以及 Recall 是否保持不变。

3.3 MPI + OpenMP + SIMD 混合并行实验

混合并行实验用于比较纯 MPI 和 MPI + OpenMP + SIMD 组合方式的性能差异。实验固定总核数为 8，设置不同的进程/线程组合：

$$(np,\ threads) = (8,1), (4,2), (2,4), (1,8).$$

其中 MPI 负责进程间数据划分和通信，OpenMP 负责 rank 内 query-level 并行，SIMD 负责距离计算加速。

该实验主要比较不同组合下的 wall latency、local search time 和 communication time。通过该实验可以判断在固定总计算资源下，增加线程数是否能抵消减少 MPI rank 后带来的本地分片增大问题。

3.4 通信 API 与负载均衡优化实验

通信 API 实验用于比较 blocking 和 nonblocking 两种通信方式对查询性能的影响。实验保持相同的算法参数和进程数，分别记录 broadcast、gather、search 和 total latency，观察非阻塞通信是否能够隐藏部分通信开销。

负载均衡实验用于分析不同分片策略对 MPI-IVF 查询性能的影响。本文比较 contiguous、round-robin 和 greedy 等策略，重点记录 max-rank search time 与 avg-rank search time。若二者差距较大，说明 rank 间搜索负载不均，整体延迟会受到最慢 rank 的限制。

3.5 MPI-IVF-PQ 压缩索引实验

MPI-IVF-PQ 实验用于分析 PQ 压缩编码在 MPI-IVF 框架下的效果。该方法先使用 PQ 编码进行粗排，筛选出一定数量的候选，再使用原始 float 向量进行 SIMD rerank，以提升最终 Recall。

实验主要控制 *nprobe* 和 *rerank_p* 两个参数。*nprobe* 控制访问的 IVF 倒排表数量，*rerank_p* 控制进入精排阶段的候选数量。本文通过不同参数组合下的 Recall 和 wall latency，分析压缩索引在精度和效率之间的折中关系。

3.6 MPI 图索引并行实验

MPI 图索引实验用于探索 HNSW 类方法在多进程环境下的并行化效果。本文主要测试三类图索引方案：MPI shard-HNSW、IVF-HNSW 和 centroid-HNSW + IVF-HNSW。

其中 shard-HNSW 将 base 向量划分到不同 rank，每个 rank 独立构建本地 HNSW 图；查询时各 rank 并行搜索本地图索引，并将 local top-k 返回 root 合并。IVF-HNSW 则先通过 IVF 选择候选簇，再在簇内 HNSW 图上搜索。centroid-HNSW + IVF-HNSW 进一步在聚类中心上建立 HNSW，用于加速 cluster 选择。

该实验主要关注 efSearch、*nprobe* 和 MPI 进程数对 Recall 与 latency 的影响。

3.7 HNSW 分片策略与节点布局实验

HNSW 分片策略实验用于分析不同数据划分方式对 shard-HNSW 性能的影响。本文比较 contiguous、round-robin、random、hash 和 kmeans-based 等策略，观察不同策略下的 Recall-Latency 分布。

节点布局实验用于分析相同总 rank 数下，不同节点分布方式对 MPI-IVF-SIMD 的影响。例如在总 rank 数固定为 8 时，比较 1×8、2×4 和 4×2 等布局。该实验重点分析节点内资源竞争与跨节点

通信之间的权衡。

3.8 dense 参数网格实验

为了更全面地展示 Recall-Latency trade-off，本文补充了 dense 参数网格实验。对于 MPI-IVF-SIMD，实验遍历不同的

$$np, \quad nlist, \quad nprobe$$

组合，并对每组参数重复运行 3 次。该实验用于绘制不同进程数下的 dense recall-latency landscape，观察 Pareto frontier 是否随进程数增加而整体左移。

对于 MPI shard-HNSW，实验遍历不同的分片策略、np 和 efSearch，分析图索引并行搜索中分片方式和搜索宽度对性能的共同影响。dense 参数网格实验不只用于寻找单个最优配置，也用于展示不同算法路线在高 Recall 和低 latency 之间的整体分布。

4 实验结果与分析

本章对各阶段实验结果进行分析。实验主要围绕 MPI-IVF-SIMD 扩展性、MPI-IVF 优化、MPI-IVF-PQ、MPI 图索引以及 dense 参数网格展开。本文重点关注 Recall@10、wall latency、speedup、parallel efficiency 和阶段耗时，从而分析不同并行策略在精度与延迟之间的权衡关系。

4.1 MPI-IVF 扩展性分析

我们首先测试 MPI-IVF-SIMD 在不同 MPI 进程数下的扩展性表现，结果如图 3 所示。

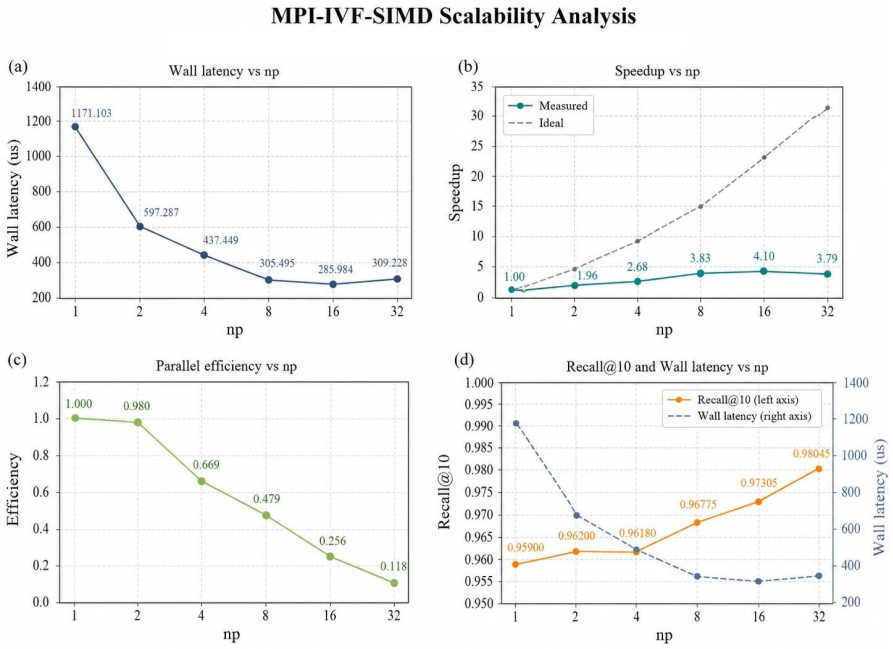


图 3: MPI-IVF-SIMD 在不同进程数下的扩展性分析

从图中可以看出，随着进程数从 1 增加到 16，wall latency 从 1171.103 μs 下降到 285.984 μs，说明 MPI 数据划分显著减少了各 rank 的本地搜索规模，能够有效提升查询速度。同时，测量加速比在 $np = 16$ 时达到最高，为 4.10×。

但当进程数继续增加到 32 时，latency 回升到 309.228 μs，并行效率也持续下降。这说明 MPI-IVF 并非进程数越多越好，当分片继续变细后，通信、同步和 root 端候选合并开销会逐渐抵消搜索规模下降带来的收益。因此，MPI-IVF-SIMD 的有效扩展区间主要位于 $np = 8$ 至 $np = 16$。

4.2 MPI-IVF 优化分析

为了减少频繁 query 广播带来的 MPI 同步开销，本文首先测试了 batch 通信优化，结果如图 4 所示。

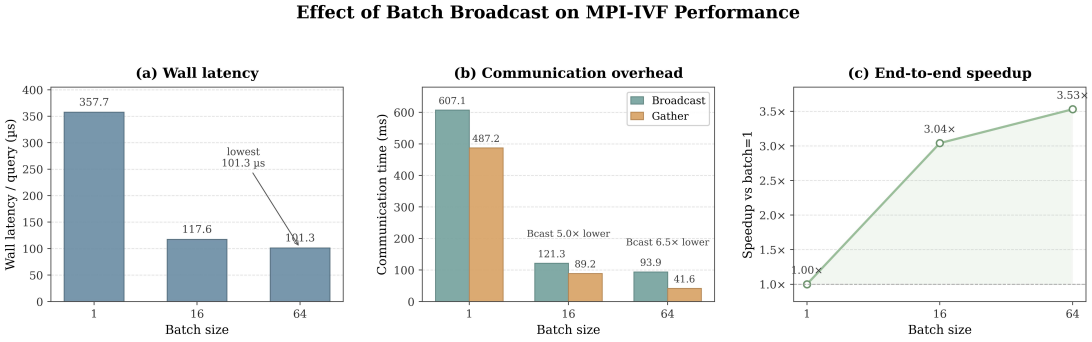


图 4: Batch 广播对 MPI-IVF-SIMD 查询性能的影响

可以看到，batch size 从 1 增加到 64 后，wall latency 从 357.7 μs 降低到 101.3 μs，端到端加速比达到 3.53×。同时，broadcast 与 gather 阶段耗时均明显下降。这说明 batch 广播能够减少 MPI 调用次数和进程同步次数，是本实验中收益最明显的通信优化之一。

在此基础上，本文测试了固定总核数为 8 时 MPI + OpenMP + SIMD 的混合并行效果，结果如图 5 所示。

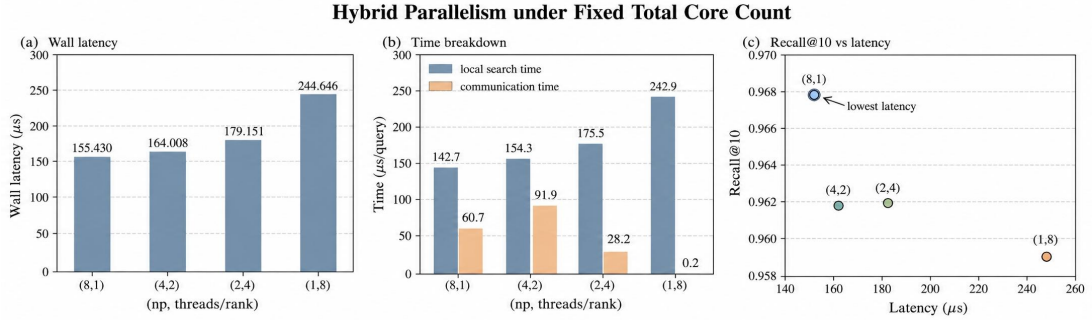


图 5: 固定总核数下 MPI + OpenMP + SIMD 混合并行性能对比

实验结果表明，纯 MPI 配置 (8, 1) 的 latency 最低，为 155.430 μs；而随着 MPI rank 减少、每个 rank 内线程数增加，整体性能反而下降。其原因在于 rank 数减少后，每个 rank 负责的 base 分片变大，本地搜索时间增加，而 OpenMP query-level 并行未能完全抵消这一额外开销。因此，在当前数据规模和实现方式下，固定总核数时纯 MPI 方案优于混合并行方案。

图 6 给出了通信 API 与负载均衡优化的实验结果。

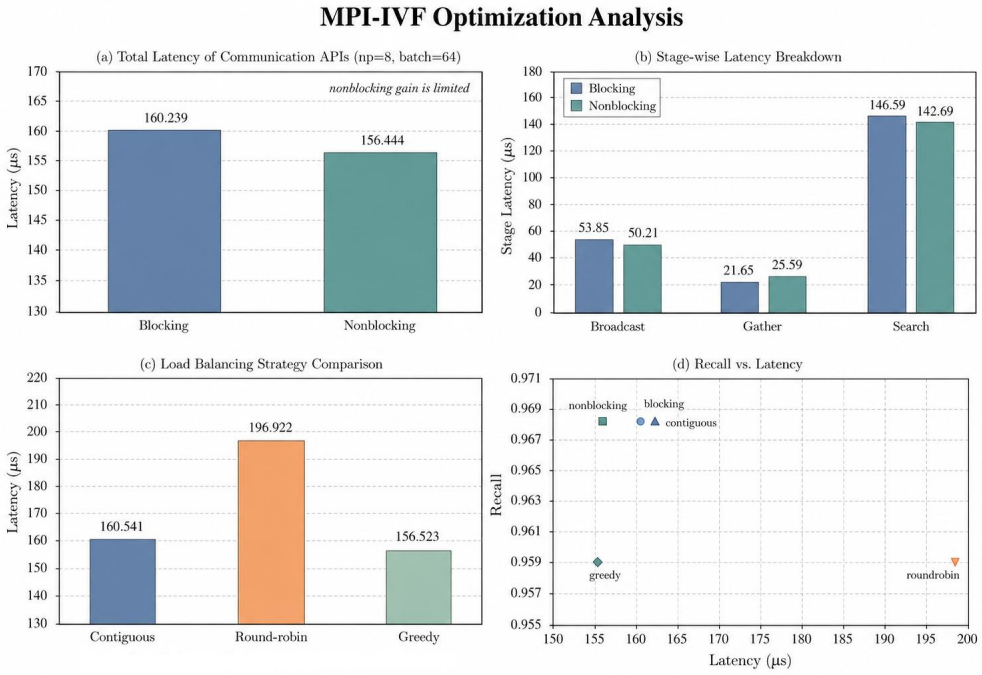


图 6: MPI-IVF-SIMD 通信 API 与负载均衡优化分析

从通信 API 对比来看，nonblocking 相比 blocking 仅将 latency 从 160.239 μs 降低到 156.444 μs，提升幅度较小。这说明在当前实现中，通信与计算重叠并不充分，主要瓶颈仍集中在本地 IVF 搜索阶段。

从负载均衡策略来看，greedy 的 latency 最低，为 156.523 μs；round-robin 最慢，为 196.922 μs。这说明简单轮转划分未必能改善搜索负载，反而可能破坏数据局部性。greedy 略优于 contiguous，但收益有限，说明当前负载不均不是最主要瓶颈。

此外，本文还测试了不同节点布局对 MPI-IVF-SIMD 的影响，结果如表 2 所示。

表 2: 不同节点布局对 MPI-IVF-SIMD 查询性能的影响

节点布局	Wall latency / μs	Broadcast / μs	Search / μs	相对加速比
1×8	165.4	54.9	159.6	1.000×
2×4	143.6	33.0	133.3	1.152×
4×2	147.7	39.3	138.4	1.120×

可以看到，在总进程数固定为 8 时，2×4 布局取得最低延迟 143.6 μs。这说明适当跨节点可以缓解单节点资源竞争，但过度跨节点也会增加通信成本，因此节点布局需要在计算资源和通信开销之间进行平衡。

4.3 MPI-IVF-PQ 结果分析

图 7 展示了 MPI-IVF-PQ-SIMD 在不同 *nprobe* 与 $rerank_p$ 下的表现。

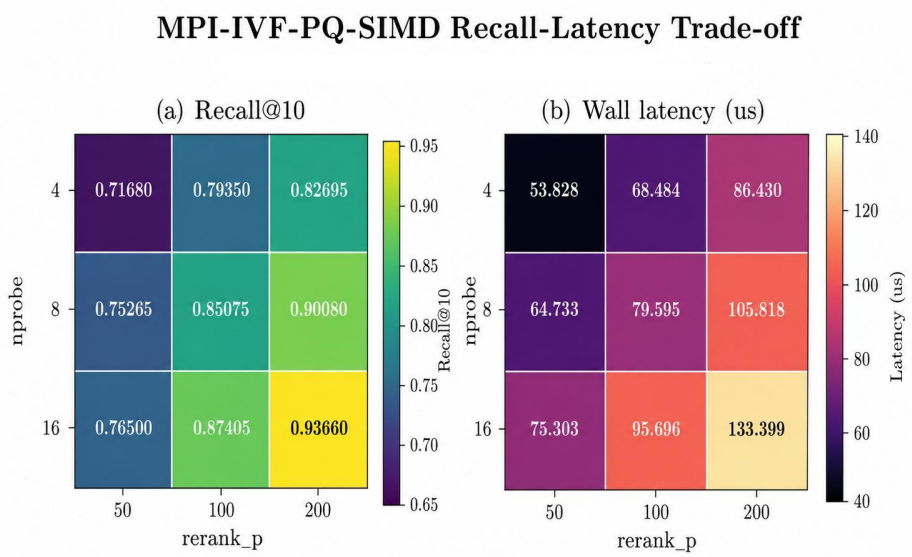


图 7: MPI-IVF-PQ-SIMD 在不同 *nprobe* 与 *rerank_p* 下的 Recall-Latency 表现

可以看到，随着 *nprobe* 和 *rerank_p* 增大，Recall@10 逐步提升，最高达到 0.93660；但 wall latency 也同步增加，最高达到 133.399 μs。这说明 PQ 压缩可以降低候选扫描和距离估计开销，但由于量化误差存在，需要通过更大的搜索范围和 rerank 数量恢复精度。

总体而言，MPI-IVF-PQ 更适合低延迟场景。如果任务对 Recall 要求较高，则需要增大 *nprobe* 与 *rerank_p*，但这会削弱 PQ 带来的延迟优势。

4.4 MPI 图索引结果分析

本文进一步测试了 MPI shard-HNSW、IVF-HNSW 以及 centroid-HNSW + IVF-HNSW 等图索引并行方案。首先，shard-HNSW 的参数结果如图 8 所示。

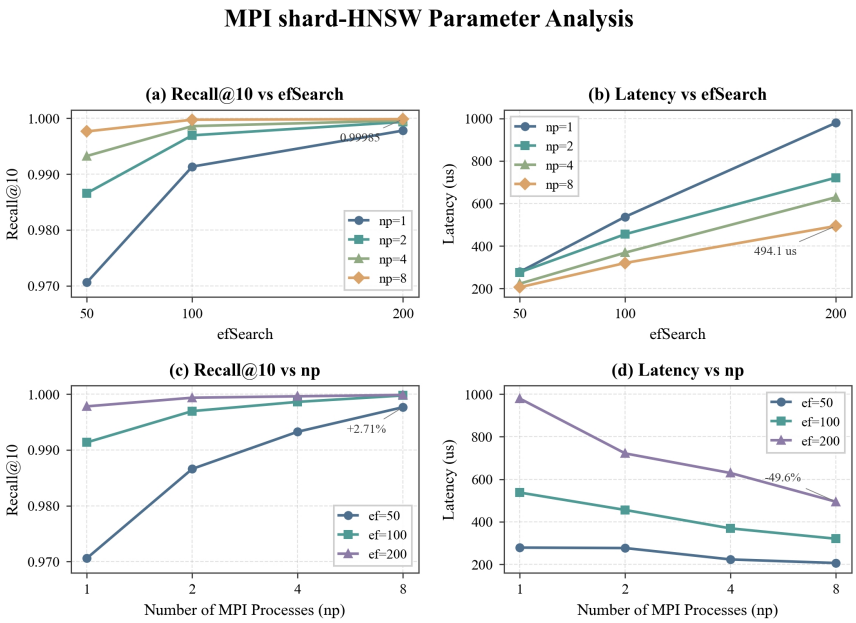
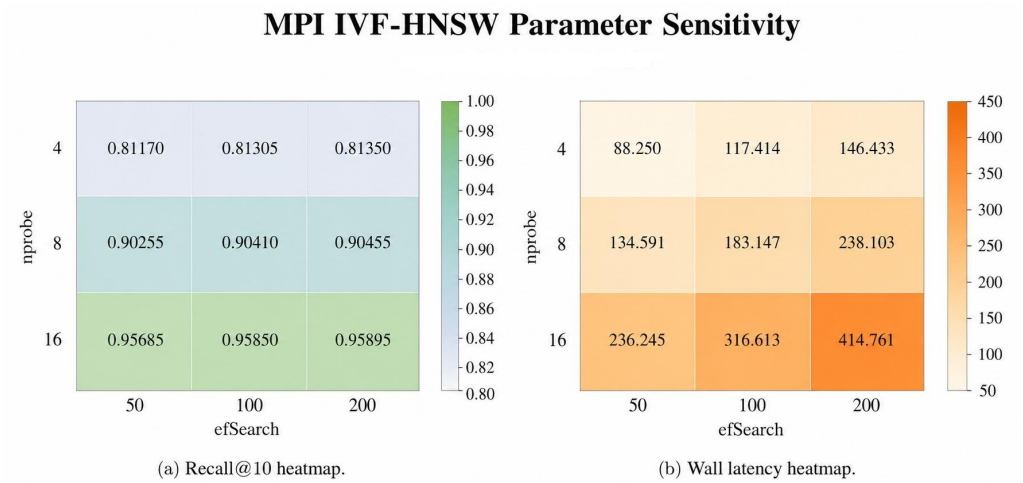


图 8: MPI shard-HNSW 在不同 efSearch 与进程数下的性能分析

结果表明，增大 efSearch 可以提升 Recall，但会带来更高 latency；增加 MPI 进程数则可以降低每个 rank 的图搜索规模，从而减少查询延迟。因此，shard-HNSW 在高 Recall 区域具有较好的潜力。

图 9 展示了 IVF-HNSW 的参数敏感性。



(a) Recall@10 heatmap. (b) Wall latency heatmap.

图 9: MPI IVF-HNSW 在不同 *nprobe* 与 efSearch 下的参数敏感性分析

从图中可以看出，*nprobe* 对 Recall 的影响更明显，而 efSearch 增大主要带来额外延迟。在 $nprobe = 16, efSearch = 200$ 时，Recall 达到 0.95895，但 latency 增加到 414.761 μs，体现了典型的精度–延迟折中。

图 10 对比了引入 centroid-HNSW 前后的结果。

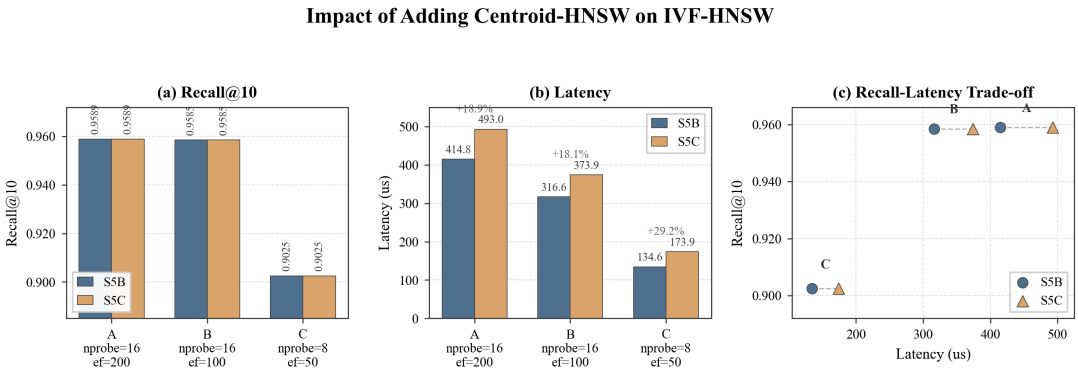


图 10: 引入 centroid-HNSW 对 IVF-HNSW 查询性能的影响

实验显示，centroid-HNSW 基本没有提升 Recall，反而增加了 latency。例如高精度配置下，latency 从 414.8 μs 增加到 493.0 μs。这说明在当前 *nlist* 规模下，直接扫描聚类中心的代价并不高，额外建立 centroid-HNSW 反而形成负优化。

综上，shard-HNSW 更适合高召回场景；IVF-HNSW 适合分析倒排索引与图索引组合后的参数影响；centroid-HNSW 在本实验规模下收益不明显。

4.5 参数网格与综合性能对比

为了更系统地观察参数对性能的影响，本文补充了 MPI-IVF-SIMD 和 MPI shard-HNSW 的 dense 参数网格实验。图 11 展示了 MPI-IVF-SIMD 在不同 *nlist*、*nprobe* 和 *np* 下的 Recall-Latency 分布。

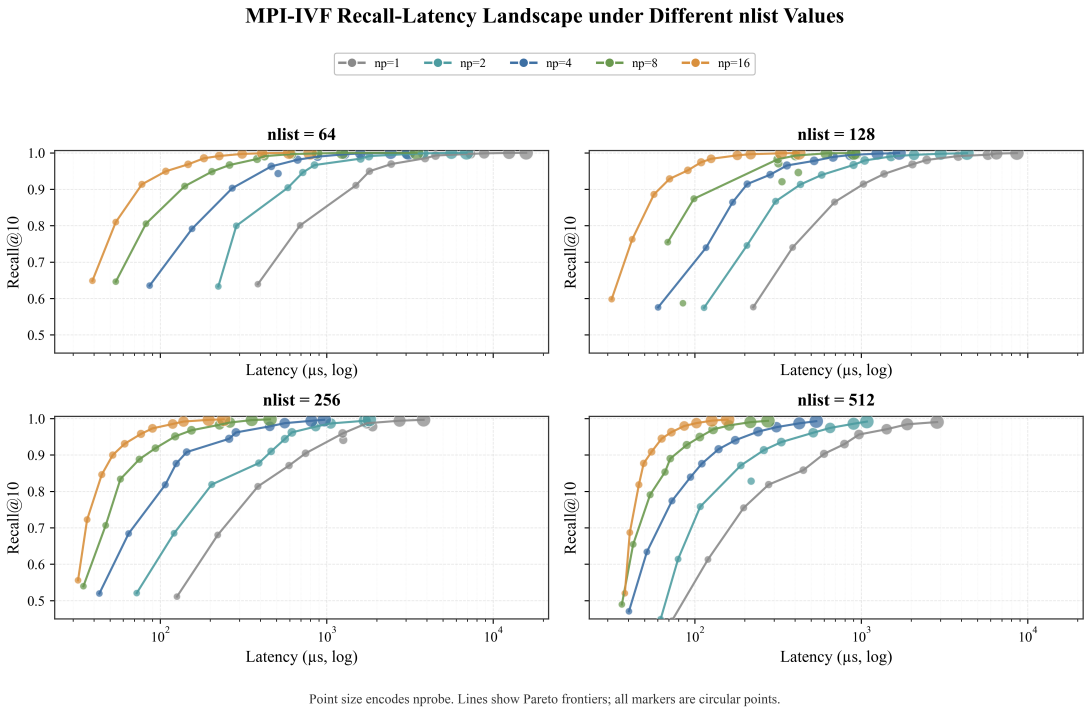


图 11: MPI-IVF-SIMD 参数网格下的 Recall-Latency 分布

可以看到，随着 *np* 增大，相同 Recall 水平下的 Pareto frontier 整体向左移动，说明更多 MPI rank 可以降低查询延迟；而 *nprobe* 增大主要提升 Recall，但也会显著增加倒排扫描规模。

图 12 展示了 MPI shard-HNSW 在不同进程数下的分片策略对比。

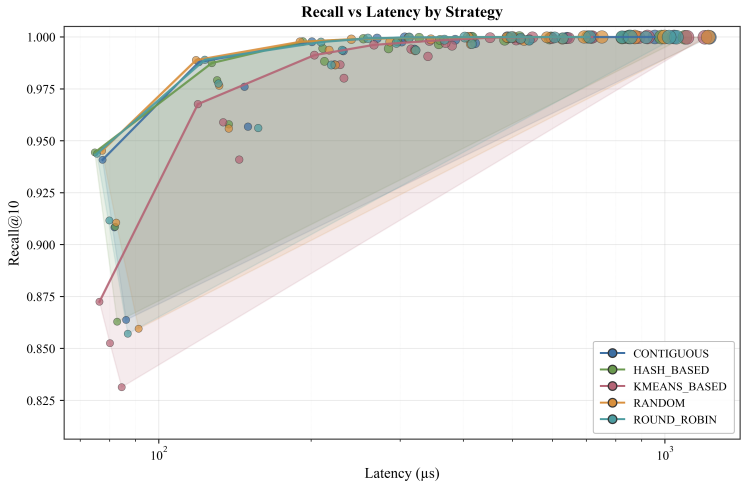


图 12: MPI shard-HNSW 不同进程数下的分片策略对比

从图中可以看出，增加进程数能够整体降低 shard-HNSW 的查询延迟；不同分片策略整体表现接近，但在低 efSearch 区间差异更明显，说明分片方式会影响局部图对全局近邻结构的覆盖能力。

最后，本文将主要方法放在统一的 Recall-Latency 坐标中进行比较，如图 13 所示。

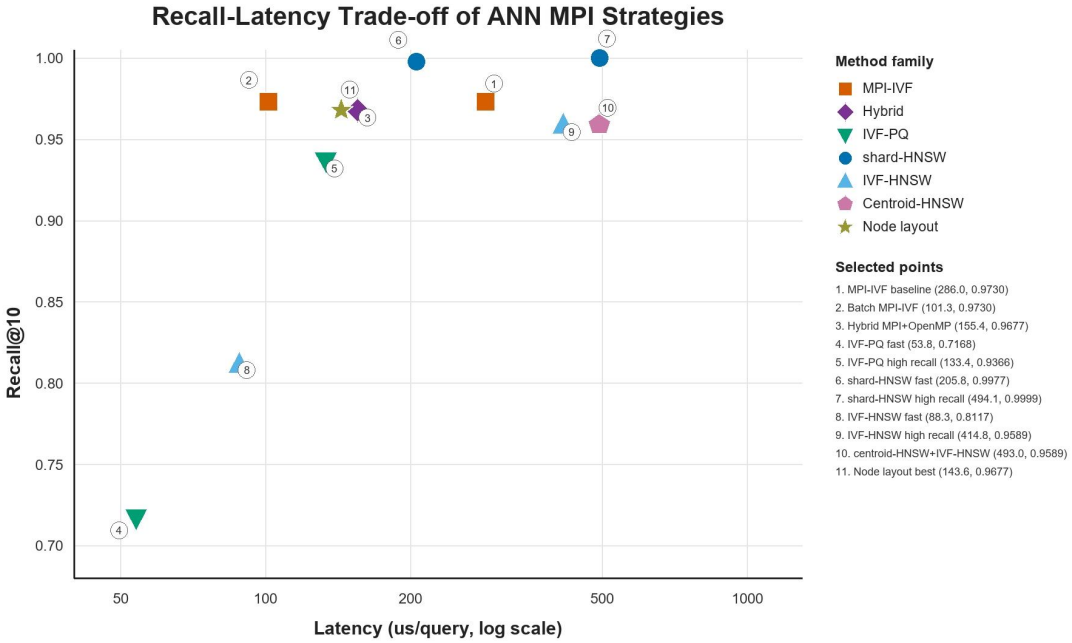


图 13: ANN 并行策略的整体 Recall-Latency 对比

综合来看，MPI-IVF-SIMD 经过 batch 优化后，在较高 Recall 下可以获得较低延迟；MPI-IVF-PQ 进一步降低了 latency，但牺牲了一定 Recall；shard-HNSW 在高 Recall 区域表现最好；IVF-HNSW 提供了倒排索引与图索引结合的折中方案；centroid-HNSW + IVF-HNSW 在当前规模下由于额外图搜索开销较大，表现为负优化。图中每个点表示对应方法在实验参数中的代表性配置。[3]

5 总结

5.1 实验结论

本实验围绕 DEEP100K 数据集上的 ANN 查询任务，在已有 SIMD 与 OpenMP 优化基础上，进一步实现并比较了 MPI-IVF-SIMD、MPI-IVF-PQ、MPI shard-HNSW、IVF-HNSW 以及 centroid-HNSW + IVF-HNSW 等多种并行搜索方案。主要结论如下：

1. **MPI 数据划分能够有效降低查询延迟**。各 rank 只需搜索本地 base 分片，显著减少了单进程扫描规模。但当进程数过多时，通信、同步和 global top-k 合并开销会逐渐抵消并行收益。

2. **batch 通信优化收益明显**。将逐 query 广播改为 query batch 广播后，MPI 调用和同步次数减少，在保持 Recall 基本不变的同时显著降低了 wall latency。

3. **固定总核数下纯 MPI 优于混合并行**。在 $(8,1)$、$(4,2)$、$(2,4)$ 和 $(1,8)$ 的对比中，纯 MPI 配置表现最好。减少 rank 后每个进程的 base 分片变大，OpenMP query-level 并行未能完全抵消本地搜索开销。

4. **不同索引方法具有不同的 Recall-Latency 权衡**。MPI-IVF-SIMD 综合表现较稳定；MPI-IVF-PQ 适合低延迟场景，但会损失部分 Recall；MPI shard-HNSW 在高 Recall 区域表现较好；centroid-HNSW + IVF-HNSW 在当前规模下未带来正收益。

[3]横轴为平均查询延迟，纵轴为 Recall@10；越靠近左上角表示方法在低延迟和高召回之间取得更优折中。

5. **dense 参数网格有助于选择合适配置**。通过对 *nlist*、*nprobe*、*np*、efSearch 和分片策略进行密集测试，可以更清楚地观察不同方法的 Pareto frontier，从而选择合适的精度–延迟折中点。

总体来看，ANN 并行搜索性能由索引结构、数据划分、通信模式和参数选择共同决定。低延迟场景可优先考虑 batch 优化后的 MPI-IVF 或 MPI-IVF-PQ，高召回场景可优先考虑 MPI shard-HNSW。

5.2 不足与改进方向

本实验仍有一些不足。首先，nonblocking 通信收益有限[4]。其次，当前负载均衡策略较简单，仍可以结合倒排表长度、图结构复杂度和历史查询统计进行更精细的划分。

5.3 个人心得体会

通过本次实验，我对 ANN 算法从单机 SIMD/OpenMP 优化到 MPI 分布式并行的完整流程有了更深入的理解。MPI 并行并不是简单增加进程数，而是需要同时考虑数据划分、通信开销、负载均衡和候选合并等问题。

实验也让我认识到，评价并行程序不能只看单一延迟指标，还需要结合 Recall、speedup、parallel efficiency 和阶段耗时综合分析。通过对 MPI-IVF、IVF-PQ 和 HNSW 等方法的对比，我对不同 ANN 索引结构的适用场景以及精度–效率折中有了更清晰的认识。

[4]这说明当前实现中通信与本地搜索计算的重叠程度不足，主要瓶颈仍集中在 local search 和结果合并阶段。

附录 A 各阶段实验参数

表 3: 各阶段代表配置、总体性能与分阶段耗时汇总

阶段	代表配置	Recall@10	Latency	Speedup	分阶段耗时 / $\mu s/query$				Comm	主要说明
			(μs)		Broadcast	Search max	Gather	Merge		
S1 MPI-IVF 基线	np=16, batch=1	0.97305	285.984	4.10×	225.616	75.994	186.746	9.660	412.362	相对 np=1。MPI 数据分片显著降低本地搜索规模，但高进程数下通信与 root merge 开销上升。
S2A batch 通信	np=16, batch=64	0.97305	101.296	3.53×	46.933	71.531	20.817	8.599	67.750	相对 batch=1。batch 广播显著减少 collective 调用次数和同步等待。
S2B 混合并行	(8, 1), batch=64	0.96775	155.430	1.00×	40.825	142.729	19.903	4.383	60.728	以 (8, 1) 为本阶段 baseline。固定 8 核下纯 MPI 最优。
S3A 通信 API	nonblocking, np=8	0.96775	156.444	1.02×	50.207	142.691	25.590	4.356	75.797	相对 blocking。非阻塞通信收益有限，通信与本地搜索没有充分重叠。
S3B 负载均衡	greedy, np=8	0.95900	156.523	1.03×	65.166	138.035	50.343	3.703	115.509	相对 contiguous。greedy 降低 search max，但收益较小，且 build time 不宜直接比较。
S4 MPI-IVF-PQ	nprobe=16, rerank=200	0.93660	133.399	1.17×	–	120.925	–	–	6.496 MB	相对 S2B IVF-SIMD (8, 1)。PQ 降低延迟，但牺牲部分 Recall。
S5A shard-HNSW	np=8, efSearch=50	0.99765	205.815	0.76×	–	–	–	–	–	相对 S2B IVF-SIMD (8, 1)。Recall 更高，但 latency 更大，适合高精度场景。
S5B IVF-HNSW	nprobe=16, efSearch=200	0.95895	414.761	0.50×	–	–	–	–	6.496 MB	相对 S5A shard-HNSW。Recall 受 IVF 粗筛限制，高精度配置下 latency 较高。
S5C centroid-HNSW	nprobe=16, efSearch=200	0.95895	492.979	0.84×	–	–	–	–	6.496 MB	相对 S5B IVF-HNSW。当前 nlist=256 下 centroid-HNSW 形成负优化。
S6 节点布局	2 × 4, np=8	0.96775	143.577	1.15×	33.015	133.256	16.937	4.337	49.952	相对 1 × 8。在单节点资源竞争与跨节点通信之间取得较好平衡。